Design Patterns: Observer

About me:

- Java Developer
- Co-Organizer of Codeus community

Observer

Observer notifies multiple objects or subscribers about any events that happen to the object they're observing.

Belongs to the Behaviour Patterns group.

Also known as: Event-Subscriber, Listener.

Observer

Key Idea: "**Don't call us, we'll call you**" – Observers register interest in the Subject and get updated via callbacks.

Main Advantages

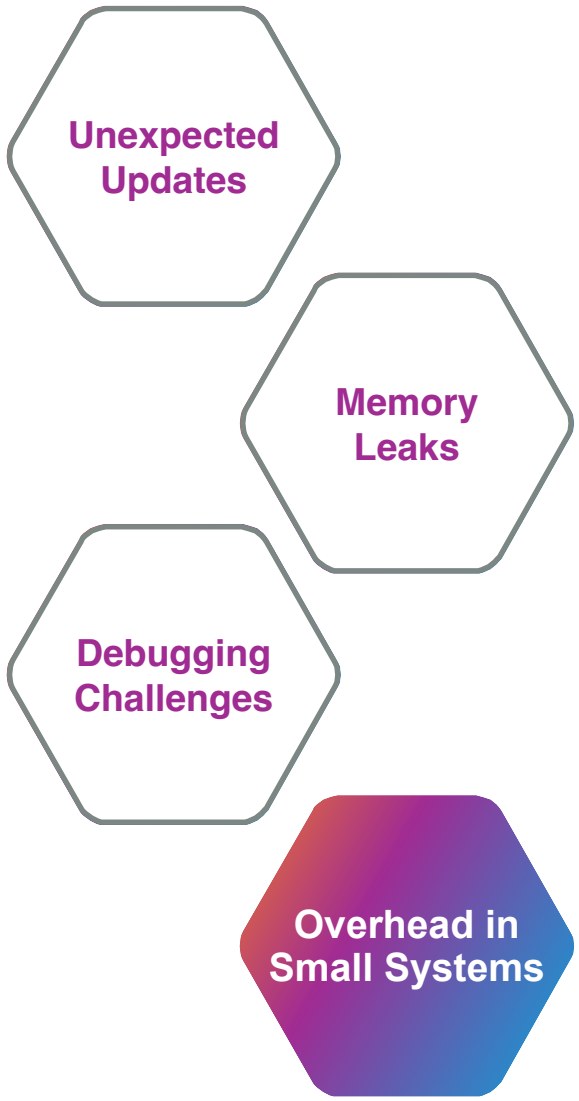
Subject and Observers are independent.
Subject doesn't need to know about specific Observers, only that they implement an interface.

Easy to add or remove Observers at runtime without modifying the Subject's code.

Efficiently notifies multiple Observers of changes in one go, reducing code duplication.

Observers can be attached/detached dynamically, supporting flexible systems.

Main Disadvantages



Observers may receive notifications for irrelevant changes, leading to unnecessary processing or performance overhead.

If Observers are not properly unregistered (detached), they can persist in memory, causing leaks in long-running applications.

Asynchronous or indirect notifications can make it hard to trace the flow of updates, especially with many Observers.

Adds complexity (interfaces, registration) that might be overkill for simple scenarios.

Where Observer Used

- GUI Frameworks

In MVC architecture, Views observe Model changes (Java Swing, .NET events)

- Event-Driven Systems

Handling user events, like button clicks notifying listeners.

- Pub-Sub Messaging

In systems like Apache Kafka or Redis Pub/Sub for distributed notifications.

- Real-World Examples

Stock market tickets. Weather stations updating multiple displays. State management

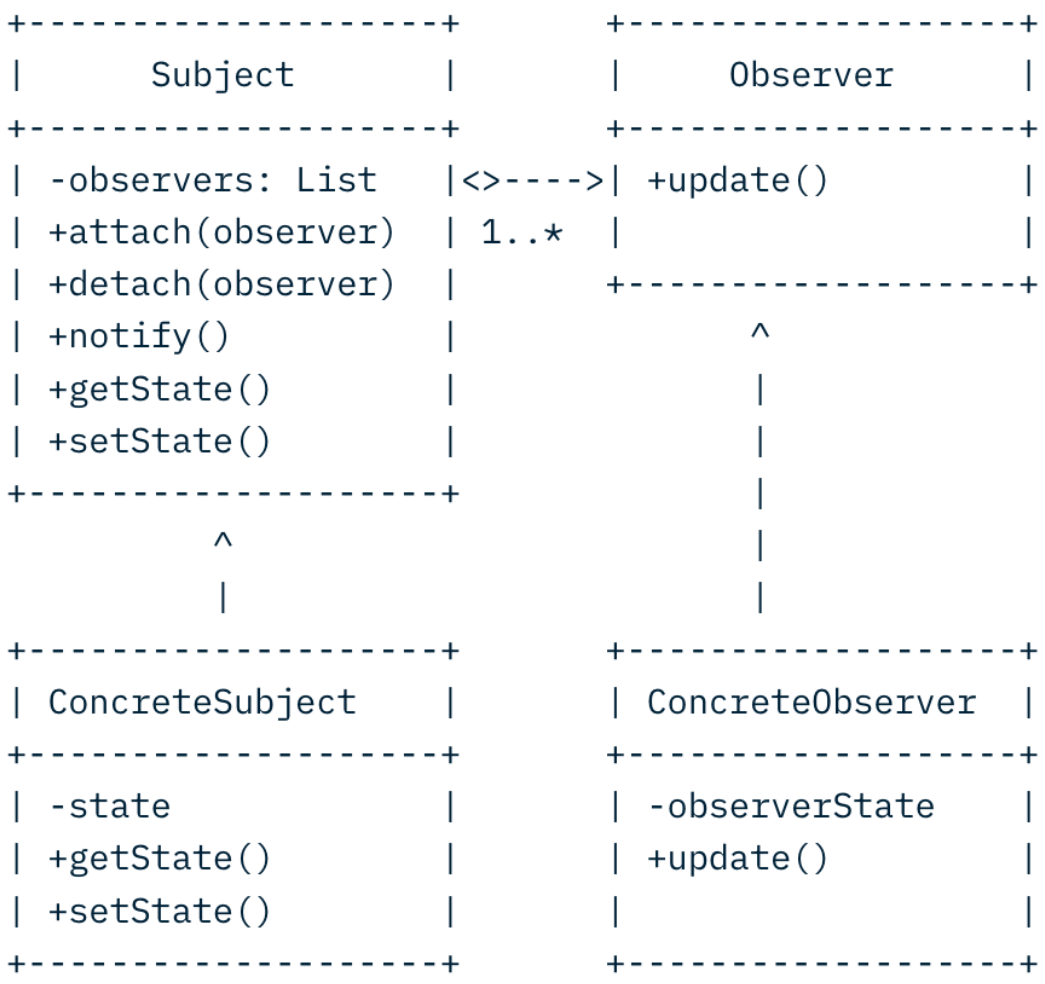
Problems Observer Solves

Eliminates tight coupling in scenarios where multiple objects need to react to a single source of change.

Handles "one-to-many" synchronisation without polling (pull model) – uses push notifications instead.

Supports scalability in dynamic environments, like multi-user applications or IoT systems where devices need to sync state.

Structure/Scheme of the Observer Pattern



Subject:
Maintains a list of Observers.
Provides attach(), detach(), and notify() methods.

Observer:
Defines an update() method that gets called by the Subject.

ConcreteSubject:
The actual object being observed; tracks state and notifies on changes.

ConcreteObserver:
Implements update() to react to notifications.

Comparison with similar patterns

Observer vs Mediator

Similarities

- Both reduce coupling between components.
- Allow objects to interact without direct references.

Differences

- **Observer**: one-to-many, Subject → Observers
- all observers receive notification
- **Mediator**: many-to-many through central mediator
- mediator decides who receives notification

When to use

- Observer: when one object changes and many others need to react.
- Mediator: when many objects interact in complex ways.

Observer vs Pub/Sub (Event Bus)

Similarities

- Asynchronous communication.
- Loose coupling between sender and receiver.

Differences

- **Observer**: direct Subject → Observer relationship.
- **Pub/Sub**: indirect relationship through Event Bus.

Key difference

- Observer knows its observers, Pub/Sub doesn't.

Observer vs Strategy

Similarities

- Allow changing behaviour at runtime.
- Avoid if/else chains.

Differences

- **Observer**: multiple reactions to one event (all observers react).
- **Strategy**: one behaviour at a time (only one strategy active).

Quantity of reactions

- Observer - many simultaneously.
- Strategy - one at a time.

✅ Use Observer when: ✅

One object changes, many need to react.

Real-time notifications are needed.

Number of observers can vary.

✕ Don’t use Observer when: ✕

Complex interaction between many objects (use Mediator).

Only one strategy behaviour needed (use Strategy).

Need to process requests sequentially (use Chain of Responsibility).

Thank you

- Author: Yevhenii Fortuna
- My LinkedIn: https://www.linkedin.com/in/yevhenii-fortuna/
- Date: September 2025
- Join Codeus community in Discord
- Join Codeus community in LinkedIn